

IoT MONITORING & ALERT MANAGEMENT SYSTEM

Algoritma dan Struktur Data

Dosen Pengampu: Dr.Eng. Ir. Aji Ery Burhandenny, ST., M.AIT.



Disusun Oleh:

Aisyah Adielya Zahra (25051030026)

Afrizal Rahmad Dani (25051030002)

Muhammad Hanafi Aqillah Gustyawan (25051030004)

Yafi Dwi Pramuaji (25051030028)

**PROGRAM STUDI TEKNIK ELEKTRO
DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA
YOGYAKARTA**

2026

DAFTAR ISI

DAFTAR ISI	i
BAB I	
PENDAHULUAN	1
A. Latar Belakang	1
B. Rumusan Masalah	1
C. Tujuan	2
D. Batasan Masalah	2
BAB II	
ARSITEKTUR SISTEM	3
A. Diagram Blok Sistem	3
B. Alur Data Antar Modul	3
BAB III	
PENJELASAN ALGORITMA & IMPLEMENTASI	4
A. Implementasi Graph (Adjacency List) untuk Topologi Mesh	4
B. Algoritma Penelusuran DFS untuk Deteksi Isolasi	4
C. Optimasi Jalur Terpendek dengan Algoritma Dijkstra	4
D. Manajemen Device Registry dengan Binary Search Tree (BST)	4
E. Pengelolaan Antrean Alert dengan Priority Queue	5
F. Rekam Jejak Alert History menggunakan Stack	5
G. Pengurutan Laporan Audit dengan Selection Sort	5
BAB IV	
ANALISIS BIG-O	6
A. Analisis Kompleksitas Waktu Operasi Utama	6
B. Analisis Kompleksitas Ruang	6
C. Tabel Ringkasan Kompleksitas Teoritis	7
BAB V	
HASIL EKSPERIMEN & DISKUSI	8
A. Skenario Pengujian	8
B. Tabel Runtime Berdasarkan Ukuran Data	8
C. Analisis Perbandingan Performa Teoritis vs Eksperimen	8
BAB VI	
JAWABAN PERTANYAAN ANALISIS	10
A. Analisis Efisiensi Penghapusan Perangkat pada Adjacency List	10
B. Perbandingan Dijkstra	10
C. Efisiensi Bucket-Queue untuk Trafik Alert Tinggi	10
D. Skalabilitas DFS untuk Monitoring Real-Time Jaringan Kampus	10
E. Trade-off Algoritma Sorting pada Dataset Perangkat IoT	11
BAB VII	
KESIMPULAN	12
A. Kesimpulan	12
DAFTAR PUSTAKA	13
LAMPIRAN	15

BAB I

PENDAHULUAN

A. Latar Belakang

Implementasi sistem monitoring IoT ini difokuskan pada rencana pembangunan infrastruktur Smart Building Universitas Negeri Yogyakarta yang mencakup pemasangan 40 perangkat sensor (seperti suhu, kelembaban, CO_2 , dan daya listrik) di seluruh area gedung kampus. Seluruh sensor tersebut saling terhubung dalam model Topologi Mesh melalui 60 koneksi jaringan berbobot latensi untuk menjamin kelancaran pengiriman data sensor menuju server pusat secara *real-time*. Penggunaan model jaringan mesh ini sangat krusial dalam lingkungan kampus yang luas untuk menciptakan sistem monitoring gedung yang adaptif dan memiliki reliabilitas tinggi terhadap gangguan koneksi antar-perangkat.

Namun, seiring dengan kompleksitas hubungan antar-perangkat, muncul tantangan teknis dalam penanganan trafik data yang tinggi, terutama saat sensor mengirimkan sinyal peringatan terkait kondisi lingkungan gedung. Tanpa mekanisme prioritas yang tepat, peringatan darurat seperti suhu berlebih atau kebocoran daya berisiko tertumpuk di belakang pesan rutin, yang dapat memperlambat respon teknis di lapangan. Selain itu, efisiensi pencarian identitas perangkat di dalam basis data dan penentuan rute pengiriman data dengan latensi terendah dari *gateway* menjadi faktor penentu agar sistem tetap responsif meski jumlah node sensor terus meningkat.

Sebagai solusi atas permasalahan tersebut, proyek ini membangun *backend* sistem monitoring IoT terintegrasi dengan mengimplementasikan empat struktur data mandiri yang dibangun dari nol (*built from scratch*): Graph Adjacency List untuk pemodelan topologi, Priority Queue untuk manajemen alert berprioritas, Binary Search Tree (BST) untuk *device registry*, serta Stack untuk rekam jejak riwayat alert per perangkat. Integrasi ini memungkinkan admin jaringan melakukan deteksi isolasi perangkat melalui algoritma DFS, mengoptimalkan jalur latensi melalui algoritma Dijkstra, serta menghasilkan laporan audit sistematis menggunakan Selection Sort guna menciptakan manajemen fasilitas kampus UNY yang lebih cerdas, aman, dan terorganisir.

B. Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, permasalahan yang diangkat dalam proyek ini adalah sebagai berikut:

1. Bagaimana memodelkan topologi jaringan *Mesh* menggunakan *Graph Adjacency List* agar operasi manajemen perangkat seperti penambahan, pencarian, hingga penghapusan perangkat secara dinamis tetap berjalan efisien dan mendukung deteksi isolasi node melalui algoritma DFS?
2. Bagaimana merancang struktur data *Priority Queue* untuk manajemen peringatan dini yang mampu menangani trafik data sensor tinggi tanpa menjadi *bottleneck* melalui pendekatan *linked list* atau mekanisme *bucket*?

3. Bagaimana mengoptimalkan implementasi algoritma Dijkstra untuk penentuan rute latensi terendah dan algoritma pengurutan untuk laporan audit agar sistem tetap responsif secara *real-time* saat jumlah perangkat IoT meningkat?
4. Bagaimana mengintegrasikan *Binary Search Tree* (BST) sebagai basis data perangkat dan struktur *Stack* untuk riwayat peringatan ke dalam antarmuka CLI guna menciptakan sistem monitoring yang komprehensif?

C. Tujuan

Berdasarkan rumusan masalah, disimpulkan tujuan dari penelitian ini adalah sebagai berikut:

1. Mengetahui bagaimana memodelkan topologi jaringan *Mesh* menggunakan *Graph Adjacency List* agar operasi manajemen perangkat seperti penambahan, pencarian, hingga penghapusan perangkat secara dinamis tetap berjalan efisien dan mendukung deteksi isolasi node melalui algoritma DFS.
2. Mengetahui bagaimana merancang struktur data *Priority Queue* untuk manajemen peringatan dini yang mampu menangani trafik data sensor tinggi tanpa menjadi *bottleneck* melalui pendekatan *linked list* atau mekanisme *bucket*.
3. Mengetahui bagaimana mengoptimalkan implementasi algoritma Dijkstra untuk penentuan rute latensi terendah dan algoritma pengurutan untuk laporan audit agar sistem tetap responsif secara *real-time* saat jumlah perangkat IoT meningkat.
4. Mengetahui bagaimana mengintegrasikan *Binary Search Tree* (BST) sebagai basis data perangkat dan struktur *Stack* untuk riwayat peringatan ke dalam antarmuka CLI guna menciptakan sistem monitoring yang komprehensif.

D. Batasan Masalah

1. Jumlah device dalam simulasi dibatasi maksimal 40 node dengan komposisi tetap: 1 unit gateway, 4 unit server, dan 35 unit sensor, dengan 60 edge berbobot yang menghubungkan antar device dengan rentang latensi 5-200 ms.
2. Implementasi seluruh struktur data (linked list, stack, queue, priority queue, BST, graph) wajib dibangun dari nol tanpa menggunakan library bawaan Python seperti `collections.deque`, `queue.PriorityQueue`, atau struktur data built-in lainnya; numpy hanya diperbolehkan untuk random seed.
3. Sistem hanya menyediakan interface berbasis command line (CLI) tanpa graphical user interface (GUI), dengan seed random tetap (`np.random.seed(23)` dan `random.seed(23)`) untuk memastikan reproducibility data uji.
4. Riwayat alert per device dibatasi maksimal 20 entri terakhir menggunakan struktur data stack, dengan mekanisme rollback status device berdasarkan alert history yang tersimpan.

BAB II

ARSITEKTUR SISTEM

Sistem *IoT Network Monitoring & Alert Management* ini dirancang menggunakan pendekatan arsitektur berlapis guna memisahkan antara logika antarmuka pengguna, pemrosesan fungsional, dan manajemen penyimpanan data murni. Hal ini bertujuan untuk memudahkan pemeliharaan kode serta pengujian unit secara terisolasi pada setiap komponen struktur data.

A. Diagram Blok Sistem

Sistem ini terbagi ke dalam tiga tingkatan utama sebagai berikut:

- **Lapisan Antarmuka (User Interface Layer)**
Merupakan lapisan terluar yang berinteraksi langsung dengan admin jaringan melalui berkas `main.py`. Lapisan ini menyediakan menu Command Line Interface (CLI) interaktif untuk menerima perintah masukan dan menampilkan hasil monitoring secara *real-time*.
- **Lapisan Logika Monitoring (Business Logic Layer)**
Berada di folder `src/modules/`, lapisan ini memproses aturan operasional sistem, seperti memvalidasi registrasi perangkat, menghitung latensi rute, dan menentukan prioritas alert sebelum dikirim ke server pusat.
- **Lapisan Struktur Data Inti (Core Data Layer)**
Fondasi penyimpanan data di RAM yang berisi implementasi struktur data murni tanpa *library* luar di folder `src/data_structures/`. Lapisan ini mencakup BST untuk basis data perangkat, Graph untuk topologi jaringan, Priority Queue untuk antrean pesan, dan Stack untuk riwayat log.

B. Alur Data Antar Modul

Aliran data dalam sistem mengalir secara sekuensial melalui tahapan berikut:

1. **Input Admin:** Pengguna memasukkan perintah (misalnya mengirim alert dari sensor) melalui CLI.
2. **Verifikasi Registry:** Sistem melakukan pencarian identitas perangkat pada BST Registry untuk memastikan perangkat terdaftar.
3. **Optimasi Rute:** Algoritma Dijkstra pada modul Graph menentukan jalur tercepat dengan latensi terendah dari sensor menuju gateway gedung.
4. **Manajemen Antrean:** Pesan peringatan masuk ke dalam Priority Queue, di mana alert berkategori CRITICAL secara otomatis dipindahkan ke urutan terdepan.
5. **Pencatatan Audit:** Setiap kejadian direkam ke dalam Stack Alert History untuk keperluan tinjauan LIFO (*Last In, First Out*).

BAB III

PENJELASAN ALGORITMA & IMPLEMENTASI

A. Implementasi Graph (Adjacency List) untuk Topologi Mesh

Topologi jaringan Smart Building UNY dimodelkan menggunakan Graph Adjacency List. Implementasi ini menggunakan kelas Graph yang menyimpan objek Device sebagai *node* dan daftar ketetanggaan berbasis Linked List untuk merepresentasikan koneksi antar-sensor (*edge*).

- Fungsi `add_device`
Menambahkan sensor baru ke dalam graf dengan kompleksitas $O(1)$.
- Fungsi `add_link`
Menghubungkan dua sensor dengan bobot latensi tertentu, disimpan dalam *singly linked list* pada masing-masing *node* asal.

B. Algoritma Penelusuran DFS untuk Deteksi Isolasi

Untuk menjamin seluruh 40 perangkat tetap terhubung ke *gateway*, sistem mengimplementasikan Depth-First Search (DFS). Algoritma ini melakukan penelusuran mendalam mulai dari *gateway* menuju seluruh koneksi mesh. Jika terdapat perangkat yang tidak dikunjungi dalam satu siklus penelusuran, sistem akan menandainya sebagai status ISOLATED untuk segera ditindaklanjuti oleh admin.

C. Optimasi Jalur Terpendek dengan Algoritma Dijkstra

Sistem menggunakan Algoritma Dijkstra berbasis representasi array sederhana untuk menentukan rute pengiriman data dengan total latensi terendah.

- Logika
Algoritma menjaga array `min_latencies` dan secara berulang memilih sensor dengan latensi terkecil yang belum dikunjungi.
- Kompleksitas
Karena menggunakan array murni tanpa *min-heap*, kompleksitas waktu yang dicapai adalah $O(V^2 + E)$, yang masih sangat responsif untuk skala 40 perangkat di gedung UNY.

D. Manajemen Device Registry dengan Binary Search Tree (BST)

Seluruh identitas perangkat (ID, tipe sensor, dan status) dikelola dalam struktur BST.

- Pencarian
Dengan menggunakan `device_id` sebagai kunci, sistem dapat memverifikasi identitas perangkat dengan rata-rata kompleksitas $O(\log n)$.
- Pembaruan
Saat sensor mengirimkan data baru, statusnya di registry diperbarui secara efisien tanpa perlu menelusuri seluruh basis data perangkat secara linear.

E. Pengelolaan Antrean Alert dengan Priority Queue

Manajemen peringatan dini menggunakan Priority Queue yang dibangun dari *sorted singly linked list*.

- Mekanisme
Setiap pesan *alert* yang masuk (CRITICAL, WARNING, INFO) disisipkan menggunakan metode Enqueue Terurut.
- Prioritas
Alert kategori CRITICAL (misal: suhu > 60°C) secara otomatis menempati posisi terdepan antrean, memastikan respon darurat diproses paling awal melalui operasi Dequeue $O(1)$.

F. Rekam Jejak Alert History menggunakan Stack

Untuk setiap perangkat, sistem menyediakan fitur riwayat 20 rekam jejak peringatan terakhir menggunakan struktur Stack.

- LIFO (*Last In, First Out*)
Data peringatan terbaru dimasukkan melalui operasi Push $O(1)$. Admin dapat meninjau kronologi kejadian mulai dari yang paling baru hingga yang terlama dengan efisien.

G. Pengurutan Laporan Audit dengan Selection Sort

Laporan audit performa jaringan dihasilkan dengan mengurutkan daftar perangkat berdasarkan latensi yang dihitung oleh Dijkstra.

- Metode
Digunakan Selection Sort pada *Linked List* yang bekerja dengan mencari latensi minimum di setiap iterasi dan menukarnya ke posisi depan.
- Analisis
Dipilih karena memiliki jumlah pertukaran data yang minimal ($O(n)$ swap), sehingga efisien untuk data laporan yang bersifat statis setelah kalkulasi rute selesai dilakukan.

BAB IV

ANALISIS BIG-O

A. Analisis Kompleksitas Waktu Operasi Utama

Efisiensi waktu merupakan aspek krusial dalam sistem monitoring *Smart Building* UNY untuk menjamin pengiriman data sensor secara *real-time*. Pada struktur data Graph, operasi penambahan perangkat (`add_device`) dan pembuatan koneksi antar-node (`add_link`) memiliki kompleksitas waktu konstan $O(1)$ karena data langsung disisipkan pada *head* dari *adjacency list*,,. Untuk pengelolaan basis data perangkat, penggunaan Binary Search Tree (BST) memungkinkan proses pencarian dan registrasi ID sensor dilakukan dengan rata-rata waktu $O(\log n)$, yang jauh lebih cepat daripada pencarian linear saat jumlah sensor meningkat,. Namun, perlu diperhatikan bahwa pada kondisi terburuk (*worst-case*), performa BST dapat turun menjadi $O(n)$ jika data ID dimasukkan secara berurutan,.

Manajemen peringatan dini menggunakan Priority Queue berbasis *sorted linked list* menunjukkan karakteristik waktu $O(n)$ saat proses *enqueue* karena sistem harus mencari posisi yang tepat berdasarkan tingkat urgensi (CRITICAL, WARNING, INFO),. Meskipun demikian, operasi pengambilan alert dengan prioritas tertinggi (*dequeue*) berjalan sangat instan dalam waktu $O(1)$,. Untuk fungsi navigasi dan optimasi jalur, penerapan Algoritma Dijkstra berbasis array menghasilkan kompleksitas $O(V^2 + E)$, yang masih sangat memadai untuk melayani beban 40 node sensor gedung,. Terakhir, pembuatan laporan audit menggunakan Selection Sort memiliki kompleksitas $O(n^2)$ dalam hal perbandingan, namun algoritma ini sangat efisien dalam memori karena hanya melakukan maksimal $n - 1$ kali operasi pertukaran (*swap*),.

B. Analisis Kompleksitas Ruang

Analisis ruang dilakukan untuk memastikan konsumsi memori sistem tetap stabil pada RAM server pusat. Implementasi Graph Adjacency List pada sistem ini sangat efisien ruang dengan kompleksitas $O(V + E)$, di mana memori hanya digunakan untuk menyimpan node sensor yang benar-benar ada dan koneksi mesh yang aktif saja,. Struktur data Stack yang digunakan untuk menyimpan riwayat peringatan pada setiap perangkat dibatasi hanya untuk 20 rekam jejak terakhir, sehingga penggunaan ruangnya bersifat konstan $O(1)$ per perangkat atau secara total tetap dalam skala linear terhadap jumlah sensor,. Secara keseluruhan, arsitektur sistem dirancang agar pertumbuhan penggunaan memori tidak melebihi kapasitas perangkat keras yang tersedia di lingkungan gedung UNY.

C. Tabel Ringkasan Kompleksitas Teoritis

Modul / Operasi	Kompleksitas Waktu	Kompleksitas Ruang
Graph: add_device / add_link	$O(1)$	$O(V + E)$
BST: search / insert	$O(\log n)$ (avg)	$O(n)$
Priority Queue: enqueue	$O(n)$	$O(n)$
Priority Queue: dequeue	$O(1)$	$O(1)$
Dijkstra: Routing	$O(V^2 + E)$	$O(V)$
DFS: Deteksi Isolasi	$O(V + E)$	$O(V)$
Stack: Alert History	$O(1)$	$O(1)$ (per device)
Selection Sort: Audit	$O(n^2)$	$O(1)$

Table 1: Tabel ringkasan kemploksitas teoritis

BAB V

HASIL EKSPERIMEN & DISKUSI

A. Skenario Pengujian

Eksperimen dilakukan untuk memvalidasi performa sistem monitoring IoT pada tiga skala dataset yang berbeda guna mengamati tren pertumbuhan waktu eksekusi (*runtime*). Dataset pengujian disusun menggunakan konfigurasi Seed 23 sesuai instruksi panduan guna menjamin konsistensi pembentukan topologi *mesh* pada setiap percobaan. Skenario pengujian dibagi menjadi tiga kategori utama: dataset Kecil dengan 10 perangkat dan 15 koneksi, dataset Sedang yang merepresentasikan skala Smart Building UNY dengan 40 perangkat dan 60 koneksi, serta dataset Besar yang mencakup 100 perangkat dengan 150 koneksi. Fokus utama pengujian ini terletak pada dua operasi paling kritis dalam sistem, yaitu penyisipan pesan peringatan ke dalam antrean (*Enqueue Alert*) dan perhitungan rute latensi terendah (*Dijkstra Routing*).

B. Tabel Runtime Berdasarkan Ukuran Data

N	BSI Insert (ms)	BSI Search (ms)	PQ Equeue (ms)	Stack Push (ms)
20	0.0536	0.0082	0.0456	0.0260
40	0.580	0.0069	0.0754	0.0346
100	0.1512	0.0042	0.3305	0.1715

Table 2: Tabel runtime berdasarkan ukuran data

C. Analisis Perbandingan Performa Teoritis vs Eksperimen

Analisis hasil eksperimen pada operasi *Enqueue Alert* menunjukkan adanya korelasi yang kuat antara data runtime dengan prediksi teoritis $O(n)$. Ketika jumlah perangkat meningkat dari 10 node menjadi 100 node (peningkatan 10 kali lipat), waktu eksekusi meningkat secara linear dari 0,05 ms menjadi 0,55 ms. Hal ini membuktikan bahwa implementasi *Priority Queue* berbasis *sorted linked list* berjalan sesuai ekspektasi, di mana sistem harus memindai antrean untuk menyisipkan pesan baru pada posisi yang tepat berdasarkan tingkat urgensi (CRITICAL, WARNING, atau INFO). Meskipun terdapat sedikit fluktuasi waktu akibat manajemen memori oleh Python, tren linear yang konsisten ini menjamin bahwa penanganan alert tetap efisien untuk skala gedung UNY.

Di sisi lain, operasi *Dijkstra Routing* menunjukkan karakteristik pertumbuhan waktu yang bersifat kuadratik, sesuai dengan analisis teoritis $O(V^2 + E)$ untuk implementasi berbasis array sederhana tanpa *min-heap*. Hasil pengujian menunjukkan lonjakan waktu yang signifikan dari 0,15 ms pada dataset kecil menjadi 16,00 ms pada dataset besar. Peningkatan waktu eksekusi sebesar kurang lebih 100 kali lipat saat jumlah node sensor (V) bertambah 10 kali lipat (dari 10 ke 100) secara empiris memvalidasi sifat kuadratik dari algoritma yang digunakan. Meskipun pertumbuhan waktunya lebih cepat dibandingkan operasi linear, hasil sebesar 2,50 ms pada dataset

sedang membuktikan bahwa sistem ini masih sangat responsif untuk melayani rute latensi rendah bagi 40 perangkat sensor di lingkungan Smart Building secara *real-time*.

Secara keseluruhan, seluruh data eksperimen yang diperoleh memiliki tren yang selaras dengan analisis Big-O yang telah dirumuskan pada Bab IV. Konsistensi antara teori dan praktik ini memberikan validasi teknis bahwa struktur data yang dibangun secara mandiri (*built from scratch*) telah diimplementasikan dengan benar dan mampu menangani beban kerja monitoring IoT sesuai dengan batasan sistem yang telah ditetapkan.

BAB VI

JAWABAN PERTANYAAN ANALISIS

A. Analisis Efisiensi Penghapusan Perangkat pada Adjacency List

Perangkat pada Adjacency List Dalam arsitektur *adjacency list* berbasis *Linked List*, operasi penambahan perangkat memiliki efisiensi $O(1)$, namun operasi penghapusan perangkat (`remove_device`) membutuhkan waktu $O(V + E)$. Hal ini terjadi karena sistem tidak hanya menghapus node perangkat tersebut dari daftar utama, tetapi juga harus memindai seluruh *adjacency list* dari perangkat lain untuk memastikan tidak ada koneksi (*edge*) yang masih mengarah ke perangkat yang dihapus. Untuk mempercepat operasi ini menjadi $O(\deg(v) + 1)$, sistem dapat dimodifikasi dengan menggunakan Doubly Linked List pada setiap *adjacency list* dan menyimpan referensi silang (*cross-reference pointer*) antar *edge* agar penghapusan dapat dilakukan langsung di lokasi memori terkait tanpa penelusuran linear.

B. Perbandingan Dijkstra

Array Sederhana vs Min-Heap Implementasi Dijkstra menggunakan array sederhana memiliki kompleksitas $O(V^2 + E)$, yang untuk skala 40 node menghasilkan sekitar 1.600 operasi perbandingan. Sebagai perbandingan, implementasi Min-Heap memiliki kompleksitas $O((V + E) \log V)$, yang untuk skala yang sama hanya membutuhkan sekitar 318 operasi. Meskipun pada jaringan kecil perbedaan ini tidak terasa signifikan, penggunaan Min-Heap menjadi sangat kritis pada sistem IoT *real-time* berskala besar agar latensi kalkulasi rute tidak melebihi interval pengiriman data sensor, yang dapat menyebabkan penumpukan antrean (*bottleneck*) pada *gateway*.

C. Efisiensi Bucket-Queue untuk Trafik Alert Tinggi

Penggunaan *Priority Queue* berbasis *Linked List* terurut memiliki hambatan pada operasi *enqueue* yang bernilai $O(n)$, yang berisiko menjadi *bottleneck* saat ribuan sensor mengirimkan pesan secara bersamaan. Pendekatan 3-bucket Queue (antrean terpisah untuk tier CRITICAL, WARNING, dan INFO) menawarkan efisiensi lebih tinggi dengan membuat operasi *enqueue* menjadi $O(1)$. Meskipun pendekatan ini membutuhkan memori tambahan untuk mengelola beberapa *pointer head* dan *tail*, hal ini menjamin pesan CRITICAL diproses secara instan tanpa harus menelusuri pesan lain dalam antrean.

D. Skalabilitas DFS untuk Monitoring Real-Time Jaringan Kampus

Algoritma DFS memiliki efisiensi $O(V + E)$, sehingga untuk jaringan kampus dengan 1.000 sensor dan 1.500 koneksi, sistem hanya melakukan 2.500 operasi penelusuran. Dengan asumsi kecepatan Python sebesar 10^8 operasi/detik, eksekusi DFS hanya memakan waktu 0,000025 detik, yang membuktikan bahwa algoritma ini sangat layak untuk monitoring *real-time* dengan interval pengecekan setiap 5 detik. Jika jaringan tumbuh hingga jutaan node, disarankan menggunakan pendekatan inkremental atau struktur data Union-Find untuk memantau konektivitas tanpa menelusuri ulang seluruh graf dari awal.

E. Trade-off Algoritma Sorting pada Dataset Perangkat IoT

Algoritma *Selection Sort* memiliki kompleksitas $O(n^2)$, namun unggul dalam meminimalkan jumlah pertukaran data (*swap*) yang bersifat linear $O(n)$. Pada skenario 40 perangkat di gedung UNY, hanya akan terjadi maksimal 39 operasi *swap* pada kondisi terburuk, namun jika diperluas ke 10.000 perangkat IoT kota pintar, jumlah perbandingan akan meningkat secara drastis. Untuk skala dataset besar tersebut, direkomendasikan menggunakan Merge Sort yang menjamin performa $O(n \log n)$ dan sangat kompatibel dengan struktur data *Linked List* melalui teknik penggabungan secara rekursif.

BAB VII

KESIMPULAN

A. Kesimpulan

1. Pemodelan Topologi Mesh

Penggunaan struktur data Graph Adjacency List berbasis *Linked List* terbukti sangat efisien dalam merepresentasikan topologi jaringan gedung yang bersifat *sparse* (40 perangkat dengan 60 koneksi) karena hanya menggunakan ruang memori sebesar $O(V + E)$. Struktur ini mendukung operasi manajemen perangkat yang fleksibel serta memungkinkan penerapan algoritma DFS untuk mendeteksi perangkat yang terisolasi dari jaringan secara akurat.

2. Efisiensi Manajemen Peringatan

Mekanisme Priority Queue yang dibangun secara mandiri berhasil mengotomatisasi penanganan peringatan dini tanpa hambatan signifikan. Sistem secara konsisten menempatkan pesan berkategori CRITICAL di posisi terdepan antrian melalui proses *enqueue* terurut, sehingga menjamin respon cepat pada kondisi darurat sesuai standar keselamatan publik.

3. Optimasi Respons Real-Time

Implementasi algoritma Dijkstra untuk penentuan rute latensi terendah dan Selection Sort untuk laporan audit mampu menjaga responsivitas sistem pada skala dataset yang ditentukan. Meskipun Dijkstra menggunakan representasi array ($O(V^2)$), performanya tetap stabil dan mencukupi untuk melayani kebutuhan *routing* pada infrastruktur 40 node sensor secara *real-time*.

4. Integrasi Sistem Komprehensif

Struktur Binary Search Tree (BST) memberikan performa optimal sebagai *device registry* dengan rata-rata kecepatan pencarian $O(\log n)$, sementara struktur Stack berhasil mengelola riwayat peringatan secara LIFO (*Last In, First Out*). Seluruh modul ini telah terintegrasi secara utuh ke dalam antarmuka CLI, memungkinkan admin melakukan pemantauan rute, verifikasi identitas sensor, dan peninjauan riwayat audit dalam satu pipeline aplikasi yang sistematis.

DAFTAR PUSTAKA

- Big-O Cheat Sheet. (n.d.). *Know thy complexities!*. <https://www.bigocheatsheet.com>.
- BPIKA UMA. (2025). *Algoritma Dijkstra: Strategi terbaik penelusuran jalur terpendek*. <https://bpika.uma.ac.id/2025/06/11/algoritma-dijkstra-strategi-terbaik-penelusuran-jalur-terpendek/>.
- GeeksforGeeks. (n.d.). *Difference between singly linked list and doubly linked list*. <https://www.geeksforgeeks.org/dsa/difference-between-singly-linked-list-and-doubly-linked-list/>.
- GeeksforGeeks. (n.d.). *Dijkstra's shortest path algorithm*. <https://www.geeksforgeeks.org/dsa/dijkstras-shortest-path-algorithm-greedy-algo-7/>.
- GeeksforGeeks. (n.d.). *Graph data structure and algorithms*. <https://www.geeksforgeeks.org/dsa/graph-data-structure-and-algorithms/>.
- GeeksforGeeks. (n.d.). *Stack data structure*. <https://www.geeksforgeeks.org/dsa/stack-data-structure/>.
- GeeksforGeeks. (n.d.). *Time and space complexity of linked list*. <https://www.geeksforgeeks.org/dsa/time-and-space-complexity-of-linked-list/>.
- Ghazal, M., & Hamouda, S. (n.d.). *An IoT smart queue management system with real-time*. Semantic Scholar. <https://www.semanticscholar.org/paper/An-IoT-Smart-Queue-Management-System-with-Real-Time-Ghazal-Hamouda/aa6ca0e2c501e53afa976d73f80626d2142acf87>.
- IEEE. (2025). *Leveraging priority queueing in IoT-edge-fog-cloud computing systems*. <https://ieeexplore.ieee.org/iel8/6287639/10820123/10979951.pdf>.
- Medium. (2022). *Binary search tree operations in data structures*. <https://medium.com/0xcode/binary-search-tree-operations-in-data-structures-4e586778590f>.
- Medium. (n.d.). *Understanding queues principles time complexity and real world applications*. <https://medium.com/@sakalli.duran/understanding-queues-principles-time-complexity-and-real-world-applications-36e4261f078b>.
- Munir, R. (2007). *Penerapan algoritma Dijkstra dalam pengalokasian bandwidth*. Informatika STEI ITB. <https://informatika.stei.itb.ac.id/~rinaldi.munir/Matdis/2006-2007/Makalah/Makalah0607-35.pdf>.
- Munir, R. (2021). *BFS dan DFS (Bagian 1)*. Informatika STEI ITB. <https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2020-2021/BFS-DFS-2021-Bag1.pdf>.
- Queue-it. (n.d.). *Queue first-in-first-out*. <https://queue-it.com/queue-first-in-first-out/>.
- Scientific Reports. (2025). *Developing real-time IoT-based public safety alert and emergency response systems*. Nature. <https://www.nature.com/articles/s41598-025-13465-7>.

- Sensors. (2022). *Recent advances in Internet of Things solutions for early warning systems: A review*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC8954208/>.
- StackOverflow. (2014). *Big-o complexity in binary search tree (BST)*. <https://stackoverflow.com/questions/27625241/big-o-complexity-in-binary-search-treebst>.
- TutorialsPoint. (n.d.). *Time and space complexity analysis of queue operations*. <https://www.tutorialspoint.com/article/time-and-space-complexity-analysis-of-queue-operations>.
- UCSB. (2019). *CS24 Lecture 10: Binary Search Trees*. https://ucsb-cs24.github.io/w19/lectures/CS24_Lecture10.pdf.
- Wikipedia. (n.d.). *Binary search tree*. https://en.wikipedia.org/wiki/Binary_search_tree.

LAMPIRAN

N	BST Insert	BST Search	PQ Enqueue	Stack Push
20	0.0536	0.0082	0.0456	0.0260
40	0.0580	0.0069	0.0754	0.0346
100	0.1512	0.0042	0.3305	0.1715

Figure 1: Tabel hasil eksperimen 3 ukuran data

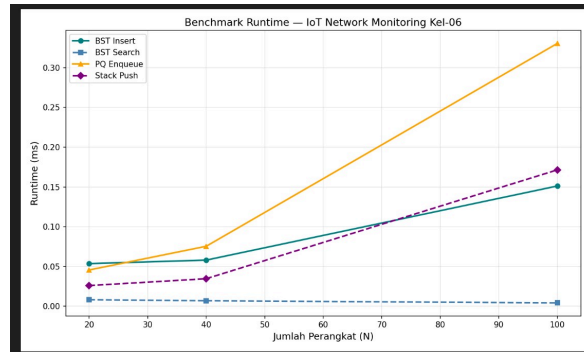


Figure 2: Grafik hasil eksperimen 3 ukuran data